15분 안에 Tripo API를 시작하고 첫 번째 3D 모델을 생성하세요.

1 API Key 발급

로 이동 API Keys 페이지에서 새 키를 생성하고 복사합니다. 안전하게 보관하세요. 한 번만 표시됩니다.

> **보안 팁:**
>
> API 키를 환경 변수에 저장합니다(TRIPO_API_KEY="sk-..." 내보내기). 소스 코드에 하드 코딩하거나 리포지토리에 커밋하지 마세요.

2 첫 번째 요청 보내기

텍스트 설명에서 3D 모델을 생성합니다. 모든 세대의 API는 비동기식입니다. 즉, 응답이 task_id를 즉시 반환합니다.

text-to-model, image-to-model, multiview-to-model에는 model을 명시해야 합니다. 시리즈를 선택하세요: H 시리즈 · P 시리즈

cURL Python JavaScript 복사

```
curl -X POST https://openapi.tripo3d.ai/v3/generation/text-to-model
\
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <token>" \
  -d '{
    "prompt": "a cute cat",
    "model": "v3.1-20260211"
  }'
```

200 응답 — 200 OK

```
{
  "code": 0,
  "data": {
    "task_id": "123e4567-e89b-42d3-a456-426614174000"
```

```
  }
}
```

표시된 작업 ID는 예시입니다. 이후 요청에는 API가 반환한 task_id를 사용하세요.

3 작업 결과 쿼리

"성공" 상태가 될 때까지 폴링하려면 task_id를 사용하세요. 2초마다 폴링합니다. 일반적인 생성에는 10~120초가 소요됩니다.

cURL | Python | JavaScript — 복사

```
# Set TASK_ID to the task_id returned by the create request
curl -X GET "https://openapi.tripo3d.ai/v3/tasks/$TASK_ID" \
  -H "Authorization: Bearer <token>"
```

200 응답 - 작업 완료


```
{
  "code": 0,
  "data": {
    "task_id": "123e4567-e89b-42d3-a456-426614174000",
    "type": "text_to_model",
    "status": "success",
    "progress": 100,
    "output": {
      "model_url": "https:// ...",
      "rendered_image_url": "https:// ..."
    }
  }
}
```


4 모델 다운로드

응답의 output.model_url에서 GLB 3D 모델 파일을 다운로드합니다. 다음에서 직접 사용할 수 있습니다.

Blender | Unity | Unreal Engine | Three.js | Babylon.js | model-viewer

> **참고:**
>
> 모델 URL는 5분 후에 만료됩니다. 작업 성공 후 즉시 다운로드하세요. 3D 모델 파일에는 output.model_url를 사용하세요.

고급 워크플로우

특정 사용 사례 및 기능에 대한 전체 API 파이프라인을 살펴보세요.

시나리오별 | 기능별

게임 준비 캐릭터
Unity/Unreal.에 사용할 수 있는 뼈대와 애니메이션이 포함된 로우 폴리 캐릭터 생성

충실도가 높은 자산
제품 시각화 또는 영화 렌더링을 위해 최고 품질의 PBR 모델을 생성합니다.

3D 프린팅
텍스처 없이 순수 지오메트리 메시를 생성하고 슬라이싱 및 인쇄를 위해 STL로 내보냅니다.

애니메이션 장면
게임 레벨이나 단편 영화에 맞게 다양한 애니메이션으로 캐릭터를 만들어 보세요.

전자상거래 제품
제품 사진을 웹 AR 체험 또는 제품 페이지용 대화형 3D 모델로 전환하세요.

멀티뷰 파이프라인
가장 정확한 3D 재구성을 위해 여러 참조 보기를 사용하십시오.

게임 준비 캐릭터

Unity/Unreal.에 사용할 수 있는 뼈대와 애니메이션이 포함된 로우 폴리 캐릭터 생성

호출 순서



API 데이터 흐름

POST **image-to-model** → POST **rig-check** → POST **rig** → POST **retarget**

1 POST /v3/generation/image-to-model

입력

Image URL or file_token

출력

```
task_id
```

→ Poll GET /v3/tasks/{task_id} until status=success

2 **POST /v3/animations/rig-check**

```
입력
{ input: "<task_id>" }
```

```
출력
task_id for rig-check
```

→ Poll the check task; if output.riggable and output.rig_type is biped, rig the original model

3 **POST /v3/animations/rig**

```
입력
{ input: "<task_id>", model: "v3.0-20260909", rig_type: "biped" }
```

```
출력
New task_id for rigged model
```

→ Poll until success, pass rigged task_id to retarget

4 **POST /v3/animations/retarget**

```
입력
{ input: "<rig_task_id>", animation: "preset:walk" }
```

```
출력
task_id
```

→ Poll until success, then download output.model_url

주요 매개변수

매개변수	가치	왜?
`model`	`P2-20260801`	게임용으로 설계된 P2로, 리깅과 애니메이션이 더 쉬운 메시를 생성
`face_limit`	`5000`	Mobile/game-friendly 다각형 개수
`texture`	`true`	캐릭터의 시각적 충실도
`model`	`v3.0-20260909`	휴머노이드에 권장되는 리그 버전(명시적으로 지정)
`rig_type`	`biped`	캐릭터용 휴머노이드 해골
`spec`	`mixamo`	Unity/Unreal 가져오기와 호환 가능
`animation`	`preset:walk`	이 예제에서 사용하는 애니메이션 프리셋 ID입니다.

비용(크레딧)

—

cURL | Python | JavaScript

복사

```python
import requests, time

HEADERS = {"Authorization": "Bearer <token>", "Content-Type":
"application/json"}

def poll(task_id):
    for _ in range(300):
        r =
requests.get(f"https://openapi.tripo3d.ai/v3/tasks/{task_id}",
headers=HEADERS, timeout=30).json()
        if r["data"]["status"] == "success": return r["data"]
```

```
        if r["data"]["status"] in ("failed", "cancelled", "banned",
"expired"): raise RuntimeError(r["data"])
        time.sleep(2)
    raise TimeoutError("Task polling timed out")

# Step 1 — Generate model
task_id = requests.post("https://openapi.tripo3d.ai/v3/generation/image-
to-model",
    headers=HEADERS, json={"input":"<file_token>","model":"P2-
20260801","face_limit":5000}
).json()["data"]["task_id"]
poll(task_id)

# Step 2 — Rig check
check_id = requests.post("https://openapi.tripo3d.ai/v3/animations/rig-
check",
    headers=HEADERS, json={"input":task_id}).json()["data"]["task_id"]
check = poll(check_id)["output"]
if not check.get("riggable"):
    raise RuntimeError("Model is not riggable")
if check.get("rig_type") ≠ "biped":
    raise RuntimeError("This character workflow requires a biped model")
rig_type = check["rig_type"]

# Step 3 — Rig
rig_id = requests.post("https://openapi.tripo3d.ai/v3/animations/rig",
    headers=HEADERS, json={"input":task_id,"model":"v3.0-
20260909","rig_type":rig_type,"spec":"mixamo"}
).json()["data"]["task_id"]
poll(rig_id)

# Step 4 — Retarget animations
anim_id =
requests.post("https://openapi.tripo3d.ai/v3/animations/retarget",
    headers=HEADERS, json={"input":rig_id,"animation":"preset:walk"}
).json()["data"]["task_id"]
```

```
result = poll(anim_id)
print("Animated model:", result["output"]["model_url"])
```

개발자 팁

- rig-check가 성공할 때까지 조회한 뒤 output.riggable과 output.rig_type을 확인하고 리깅할 수 없으면 중단하세요.
- 2초마다 작업을 폴링합니다. 속도 제한을 피하기 위해 요청 1개를 초과하지 마세요/second
- Unity/Unreal의 경우 사양: "mixamo"를 사용하세요. 커스텀 파이프라인에는 spec: "tripo"를 사용하세요.
- v3.0 리그는 preset:* 이름(preset:walk, preset:run...)을 사용하며, preset:biped:* 계열은 기본 v1.0 리그에서만 작동합니다

일반적인 함정

- rig-check가 riggable=false를 반환하는 경우 더 깔끔한 프롬프트로 다시 생성해 보십시오(복잡한 포즈는 피하십시오).
- Retarget은 조작된 모델 task_ids만 허용합니다. 원시 생성 task_id를 전달하면 실패합니다.